

GPUを用いた非圧縮性流体シミュレーションにおける データ構造 AoS と SoA に着目した性能評価

Performance Evaluation of Incompressible Fluid Simulation on GPUs focused on the
Data Structures of AoS and SoA

齊濟¹⁾小野謙二²⁾大島聡史³⁾
Ji Qi, Kenji Ono and Satoshi Ohshima

¹⁾九州大学 システム情報科学府 (〒 819-0395 福岡県福岡市西区元岡 744, E-mail: qi.ji.558@s.kyushu-u.ac.jp)

²⁾九州大学 情報基盤研究開発センター (〒 819-0395 福岡県福岡市西区元岡 744, E-mail: keno@cc.kyushu-u.ac.jp)

³⁾九州大学 情報基盤研究開発センター (〒 819-0395 福岡県福岡市西区元岡 744, E-mail: ohshima@cc.kyushu-u.ac.jp)

Computational fluid dynamics codes make use of large arrays to represent multi-dimensional vector fields. On GPUs, memory accesses can be optimized using the memory coalescing technique, however, due to the three-dimensional nature of most CFD problems, it is impossible to arrange all memory accesses into continuous blocks. In this study, we evaluate the performance of using AoS and SoA to represent multi-dimensional vector fields in an in-house incompressible fluid simulation code on multi-GPUs, and estimate the influence of these data structures on different operations within the simulation.

Key Words : CFD, HPC, GPU accelerated simulation, SoA and AoS

1. はじめに

数値流体力学 (CFD) は、微分方程式の数値解法を用いて流体の支配方程式を解き、流体の動きを解明する方法であり、風洞実験とともに様々な分野で大きな役割を持つ方法として知られている。

近年、GPU の性能向上、および開発・実行環境の整備に伴い、GPU を用いた CFD シミュレーションは増加している。しかし、不適切な実装手法を使えば、GPU の性能を十分に生かせない恐れがある。数値流体力学問題には速度などのベクトル場が多く存在し、ベクトル場の表現はプログラムのメモリー参照に大きな影響を与え、シミュレーションの性能に非常に重要だといえる。本研究は、ベクトル場の Array of Structures (AoS) と Structure of Arrays (SoA) 表現に着目し、それらのデータ構造の GPU を用いた CFD シミュレーションへの影響を評価する。

2. 非圧縮性流体シミュレーションの基礎

(1) 支配方程式

空気、水などの非圧縮性流体に対し、運動量保存を表すナビエ・ストークス方程式は

$$\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u} \cdot \nabla \mathbf{u} = -\nabla p + \frac{1}{Re} \nabla^2 \mathbf{u} \quad (1)$$

となる。

質量保存を表す連続の式は

$$\nabla \cdot \mathbf{u} = 0 \quad (2)$$

となる。

(2) 数値計算法

本研究では、圧力・速度連成法である fractional step (FS) 法 [1] を用いる。FS 法は、1 つの Δt での時間積分を 2 段階に分け、第 1 段階は、圧力の影響のない「擬似的な速度」を求める。

$$\mathbf{u}^* = \mathbf{u} + \Delta t(-\mathbf{u} \cdot \nabla \mathbf{u} + \frac{1}{Re} \nabla^2 \mathbf{u}) \quad (3)$$

第 2 段階は、擬似的な速度場に圧力を加える「projection」操作を行う。

$$\mathbf{u}_{next} = \mathbf{u}^* - \Delta t \nabla p \quad (4)$$

式 (4) に (2) を代入すると、圧力のポアソン方程式を導ける。

$$\nabla^2 p = \frac{\nabla \cdot \mathbf{u}^*}{\Delta t} \quad (5)$$

したがって、コロケート格子において、1 つの Δt での時間積分は、下記の流れになる。

Algorithm 1: FS time integration over Δt on collocated grid

- 1 Calculate $\mathbf{u}^*$;
 - 2 Interpolation $\mathbf{u}^*$ from grid points to grid faces;
 - 3 Calculate $\nabla \cdot \mathbf{u}^*$ using face values;
 - 4 Solve Poisson equation;
 - 5 Project p on both grid points and faces;
-

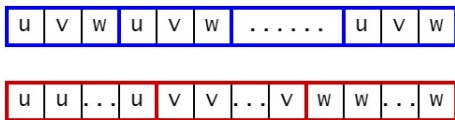


図-1 AoS（上）と SoA（下）

3. ベクトル場の SoA と AoS 表現

前述のように、FS 法には速度、擬似的な速度、および座標などのベクトル場が存在している。ベクトル場は多次元であるが、メモリは 1 次元のアドレスで管理される。多次元ベクトル場のデータ構造は、メモリでの並び方により、AoS と SoA に分けられる（図-1）。速度ベクトル $u = [u, v, w]$ を例として、AoS と SoA の構造を説明する。AoS は、1 つの計算格子の u, v, w 各成分を連続の要素で保存し、これを「structure」と呼ぶ。各格子の structure は連続に配置され、AoS になる。SoA は、 u, v, w をそれぞれ 1 つの配列で保持し、 u 配列には各計算格子の u 成分を連続に配置している。 u, v, w 3 つの配列は SoA になる。

表-1 には、FS 法各ステップが参照するデータの種類のを示す。すべてのステップは、ベクトルとスカラー 2 種類の変数をメモリから読み出す。ポアソン方程式を用いた圧力の計算は、圧力場の値を変更するため、スカラー変数をメモリに書き戻す。そのほかのステップは速度場、すなわちベクトル変数をメモリに書き戻す。

表-1 一般座標系における FS 法が参照するデータの種類の

	READ	WRITE
calculate u^*	vector, scalar	vector
interpolate u^*	vector, scalar	vector
poisson eq.	vector, scalar	scalar
project p at grid point	vector, scalar	vector
project p at grid face	vector, scalar	vector

4. GPU でのメモリ参照

(1) GPU 並列の仕組み

GPU は、大量の演算コアを持ち、多数のスレッドを同時に実行する。GPU のスレッドは並列処理の最小単位であるが、命令スケジューリングの最小単位は Warp である。Warp は 32 スレッドで構成され、それらのスレッドは常に同一命令を実行する。

GPU でのメモリ参照を効率的に行うには、下記の 2 つの点が重要である [3]：

a) Coalesced メモリ参照

Warp がメモリ参照命令を実行するとき、32 スレッドの参照先アドレスが連続であれば、その 32 回のメモリ参照を 1 回に coalescing でき、性能を大きく向上できる。

b) メモリ参照の局所性

関連性の強いデータをメモリの中で互いに近くに配置すると、GPU のキャッシュを活用でき、メモリへのアクセスを回避し、性能を向上できる。

(2) AoS と SoA へのメモリ参照

前述の coalesced メモリ参照と局所性を考えると、AoS と SoA へのメモリ参照は表-2 に示した特徴がある。

ここで、下記の命令を例としてそれらの特徴を説明する。1 つのスレッドは、1 つの計算格子の速度各成分を読み、それに基づき計算を行う。

```
GPU kernel start
i = thread_id
read u@grid_i /* command#1 */
read v@grid_i /* command#2 */
read w@grid_i /* command#3 */
/* do some calculation */
GPU kernel end
```

図-2 には、それら命令のメモリ参照のパターンを示す。ここで、便利のために、Warp のスレッド数を 3 とし、青色の部分はスレッド 1~3 が参照するメモリである。

表-2 AoS と SoA へのメモリ参照

data structure	coalescing	locality
AoS	not coalesced	local
SoA	coalesced	not local

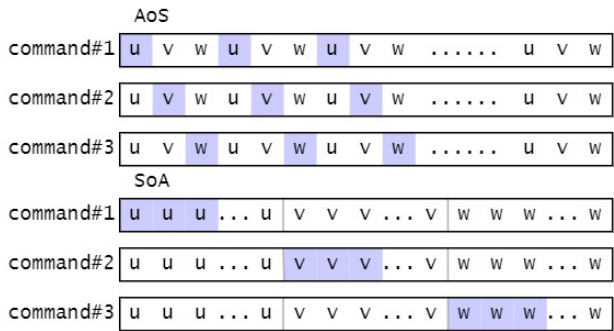


図-2 メモリ参照のパターン

AoS の場合、成分 u のメモリは連続でないため、その Warp の命令ごとのメモリ参照は coalesced ではない。その一方、格子 1~3 のデータは連続に配置されているので、その 3 つの命令は比較的に小さい範囲に参照している。

SoA の場合、各成分はそれぞれ連続のメモリに配置されているため、その Warp の各メモリ参照命令は coalescing できる。しかし、同じ格子の各成分はメモリの中に散在しているため、キャッシュのメリットを引き出すことが難しい。

(3) GPU 間通信のメモリ参照

GPU 1 基のメモリはおおむね十数 GB であり、大規模シミュレーションの場合ではメモリ不足の恐れがある。計算負荷を複数の GPU に分散するために、本

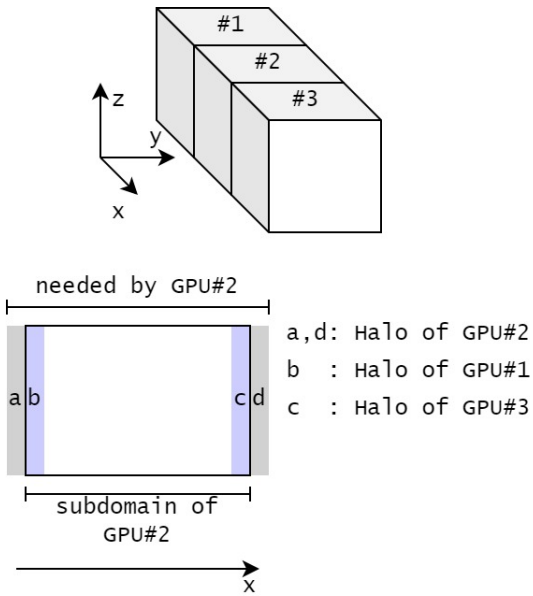


図-3 領域分割と Halo

研究は計算領域を x 軸に沿って分割し、1つのサブ領域を1基のGPUに割り当てる。

ステンシル計算において、ある格子点の計算には隣接する格子点の情報が必要であるため、分割されたサブ領域の両端に Halo と呼ばれる部分がある。Halo 部のデータは、本 GPU の計算に必要なが、Halo の計算は隣の GPU が担当している。したがって、GPU の間では、Halo 部分のデータを交換する必要がある。図-3には、GPU#2 のサブ領域と Halo を示している。

Halo	SEND	RECEIVE
a	GPU#1	GPU#2
b	GPU#2	GPU#1
c	GPU#2	GPU#3
d	GPU#3	GPU#2

表-3 Halo の送信と受信

前述のデータ構造により、GPU 間通信のためのバッファが異なる点には注意が必要となる。速度場を例としてバッファのメモリーレイアウトを説明する。図-4には、各 Halo に対応しているバッファのレイアウトを示している。AoS の場合、バッファは Halo と同じく、速度場を保持するメモリーの両端に位置する。それに対し、SoA の場合、速度の各成分はそれぞれ各自のバッファを持ち、同じ Halo に対応しているバッファが、3セグメントに分けられ、メモリーの中に散在する。したがって、AoS の場合では、2回の SEND と2回の RECEIVE が必要であり、SoA の場合では SEND と RECEIVE 各6回が必要となる。その一方、Halo のサイズはデータ構造にかかわらず、一定であるため、通信量には差がない。

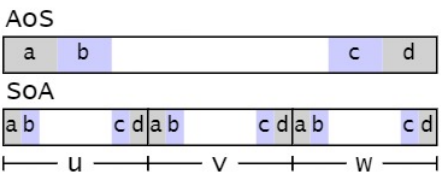


図-4 バッファのレイアウト

5. 実験による性能評価

(1) 実験環境と問題設定

性能評価の数値実験として、風車2基の風況解析を行う [2]。全計算格子数は約9千万であり、無次元時刻 $T = 10$ まで時間積分を計算する。そのシミュレーションは、九州大学スーパーコンピューター ITO のサブシステム B の1ノードで行う。システムの諸元は表-4に示す。プログラムは、NVidia HPC SDK (NVHPC) が提供した OpenMPI 環境を用いてコンパイルし、4つのプロセス (GPU 4基) で実行する。

表-4 ITO サブシステム B の1ノード

CPU	Intel Xeon Gold 6140 18 core x2
GPU	NVIDIA Tesla P100 x4
performance	CPU: 2649.6 GFLOPS/node GPU: 5.3 TFLOPS/GPU
memory	DDR4: 384 GB/node HBM2: 16 GB/GPU
CPU-GPU connect	PCIe Gen.3 x16 (16 GB/s)
GPU interconnect	NVLink (20 GB/s x1or2)

(2) 実行時間の内訳

表-5には、実行時間の中でアルゴリズム1に示した各ステップに対応するカーネルの割合を示す。

表-5 実行時間の内訳

	AoS		SoA	
	time(s)	ratio	time(s)	ratio
total GPU time	2115		1519	
calculate u^*	574	27.1%	335	22.1%
interpolate u^*	367	17.4%	131	8.6%
poisson eq.	341	16.2%	346	22.8%
project p at grid	94	4.4%	97	6.4%
project p at face	203	9.6%	102	6.7%

SoA は AoS と比べて実行時間の約 1/4 を削減し、擬似速度の計算、内挿、および格子面での projection では大きな性能改善を示している。その一方、ポアソン方程式の計算と格子点での projection では、大きな変化がないことも明らかとなった。

6. 議論

前述のように、SoA はメモリー参照を *coalescing* でき、シミュレーション性能の大きな改善を示している一方、ポアソン方程式の計算など、性能に変化のないカーネルも存在している。ここで、その原因について議論する。

SoA と AoS は、スカラー場のデータ構造に影響を及ぼさないため、スカラー場へのメモリー参照が比較的多いカーネルでは、SoA の効果を引き出しにくいと考えられる。また、メモリー参照と比べて、計算のほうが比較的に多いカーネルでは、メモリー参照の性能が改善されても、全体の性能には大きな違いがないことも推察される。

したがって、ここでは「V/S」と「B/F」という2つの値を定義する。V/S はカーネルの中で、ベクトル場に対するメモリー参照とスカラー場に対するメモリー参照の比であり、B/F はカーネルのメモリー参照の byte 数と浮動小数点演算数の比である。

表-6 V/S, B/F, および性能の比

	V/S	B/F	perf. SoA/AoS
calculate u^*	8.25	1.43	1.71
interpolate u^*	5	14	2.82
poisson eq.	1.2	4.51	0.99
project p at grid	1.5	8	0.97
project p at face	2	8	1.99

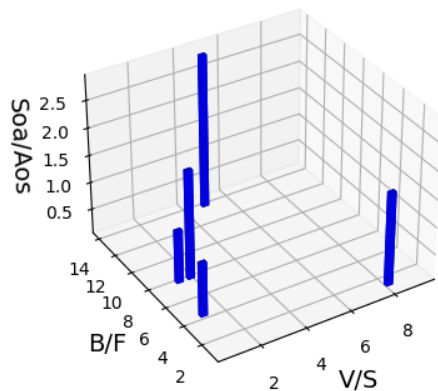


図-5 V/S, B/F, および性能の比

表-6 と図-5 には、表-5 に示した各カーネルの V/S, B/F, および SoA と AoS の性能の比を示す。図-5 に、青色の棒は各カーネルに対応し、棒の長さが SoA と AoS の性能の比を表し、棒の位置が V/S と B/F の値に対応する。表-6 と図-5 に、V/S と B/F の小さいカーネルは SoA と AoS の性能がほぼ同じであり、V/S または B/F の大きいカーネルは SoA が AoS よりいい性能を引き出せる傾向を示している。

7. 結論

本研究はマルチ GPU を用いた非圧縮性流体シミュレーションのコードに基づき、ベクトル場の AoS と SoA 表現を実装し、その性能評価を行った。

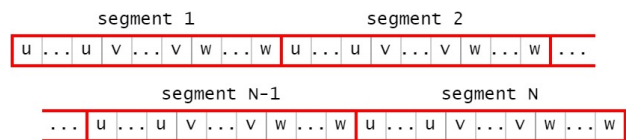


図-6 AoSoA での速度場のメモリーレイアウト

SoA と AoS はそれぞれ異なるメモリー参照のパターンを持ち、どちらも GPU メモリー参照の *coalescing* と局所性という2つの重要な点を同時に確保するのが困難であるが、シミュレーション全体としての性能は、SoA が AoS を大きく上回り、*coalesced* メモリー参照が優先であることを示した。

その一方、各カーネルの特徴により、SoA のカーネルごとの性能が異なる。V/S または B/F の高いカーネルでは SoA が大きな改善を示し、V/S と B/F の低いカーネルではその効果を引き出しにくい傾向がある。

8. 展望

本研究では、V/S と B/F などを計算するとき、WRITE（書き戻し）と READ（読み出し）を区別せず、どちらも1回のメモリー参照とする。しかし、実際には、WRITE の性能が READ を大きく下回る可能性も低くない。したがって、カーネルのメモリー参照特徴をさらに詳しく研究するには、WRITE と READ の性能の違いを考慮する必要がある。

また、メモリー参照の *coalescing* と局所性は、GPU におけるメモリー参照の性能に非常に重要な点であるが、本研究が対象としている SoA と AoS がその両方を同時に確保するのは難しい。それに対し、Array of Structures of Arrays (AoSoA)[4][5] が提案された。図-6 に示したように、ベクトル場を等長のセグメントに分け、1つのセグメントを1つの SoA に保存し、複数の SoA が連続に並び、AoSoA になる。AoSoA は *coalescing* と局所性両方を活用するポテンシャルを持つと考えられ、AoSoA によるベクトル場の表現を実装し、その性能を評価することが本研究の将来の課題となっている。

参考文献

- [1] 梶島岳夫: 乱流の数値シミュレーション, 養賢堂, 1999.
- [2] 内田孝紀, 小野謙二, 飯田明由, 吉村忍, 加藤千幸, 山出吉伸, 今村博, 植田祐子: スパコン版リアムコンパクトによる風車ウエイクの相互干渉に関する大規模数値シミュレーション, 風力エネルギー学会論文集, Vol.45(4), pp.71-82, 2021.
- [3] Cheng, J., Grossman, M., McKercher, T.: Professional CUDA C Programming, Wrox, 2014.
- [4] Fei, Y., Huang, Y.H., Gao, M.: Principles towards Real-Time Simulation of Material Point Method on Modern GPUs, arXiv:2111.00699 [cs.GR], 2021.
- [5] Wang, X.L., Qiu, Y.X., Slatery, S.R., Fang, Y., Li, M.C., Zhu, S.C., Zhu, Y.X., Tang, M., Manocha, D., Jiang C.F.F.: A massively parallel and scalable multi-GPU material point method, *ACM Transactions on Graphics*, Vol.39(4), pp.1-15, 2020.